Pico串口读取数据

Pico串口读取数据

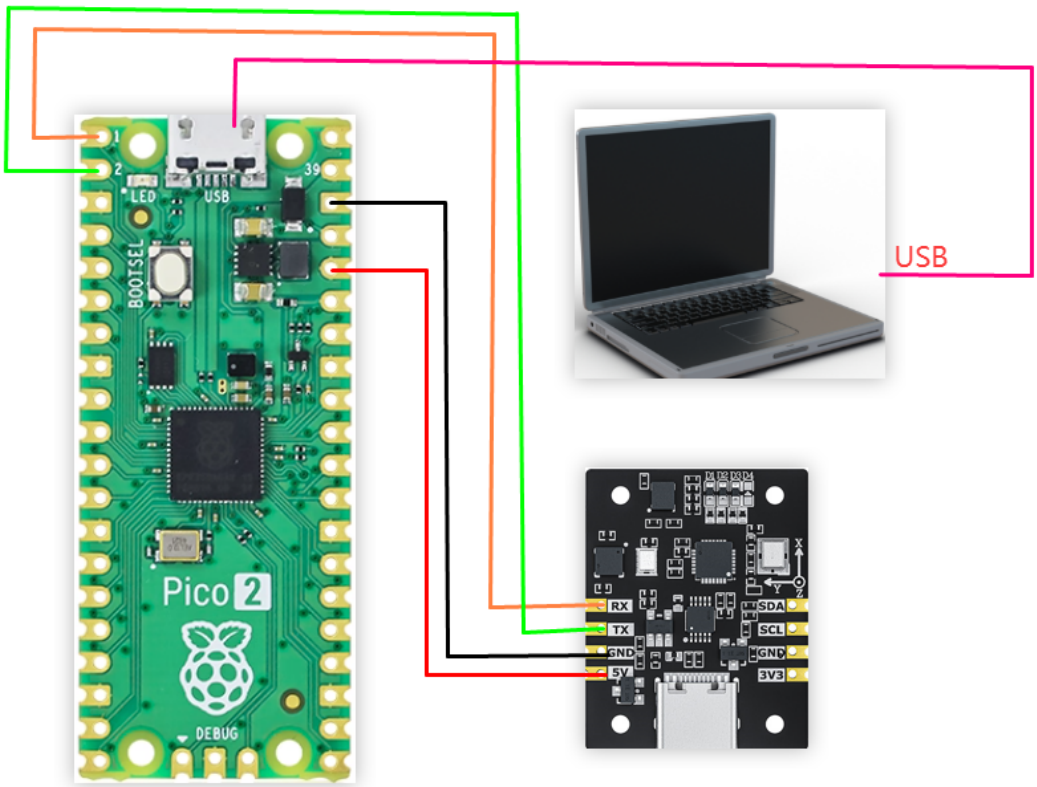
1.连接设备

2.关键代码解析

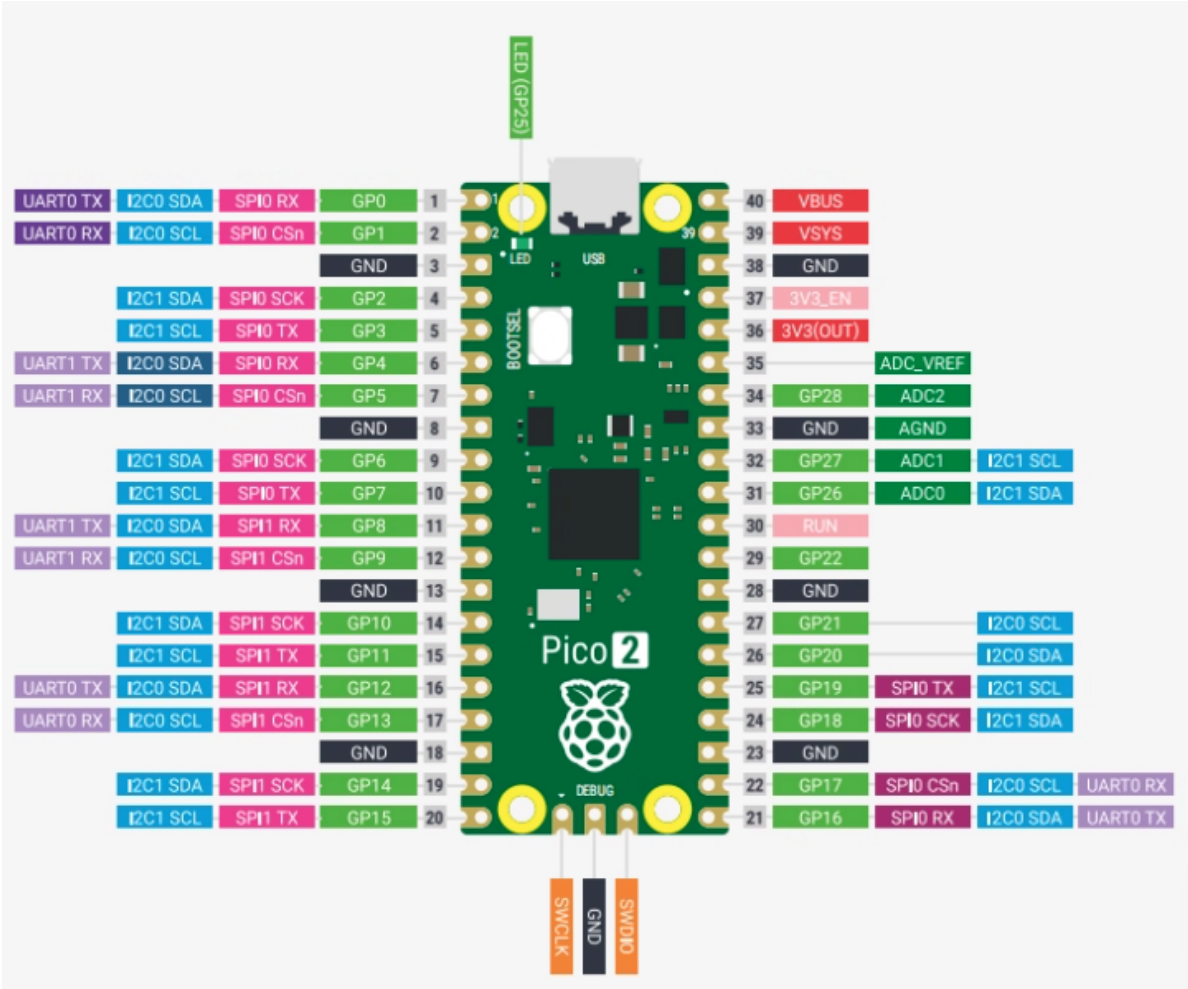
3.读取imu数据

本次例程使用的是Pico2开发板，一台windows电脑、杜邦线若干、IMU姿态传感器。

1.连接设备



IMU姿态传感器	Pico2
RX	GP0
TX	GP1
GND	GND
5V	3V3(OUT)



2.关键代码解析

具体代码请看资料中的源码。

```
# 根据数据帧的类型来做出对应的解析
    # According to the type of data frame to make the corresponding parsing
    def _parse_data(self, ext_type, ext_data):
        # print("parse_data:", ext_data, ext_type)

        # 解析原始陀螺仪、加速度计、磁力计数据
        # the original gyroscope, accelerometer, magnetometer data
        if ext_type == self.FUNC_REPORT_IMU_RAW:
            # 转化单位为g
            accel_ratio = 16 / 32767.0
            self._ax = struct.unpack('h', bytearray(ext_data[0:2]))
[0]*accel_ratio
            self._ay = struct.unpack('h', bytearray(ext_data[2:4]))
[0]*accel_ratio
            self._az = struct.unpack('h', bytearray(ext_data[4:6]))
[0]*accel_ratio

            # 转化单位为rad/s
            AtoR = math.pi / 180.0
            gyro_ratio = (2000 / 32767.0) * AtoR
            self._gx = struct.unpack('h', bytearray(ext_data[6:8]))
[0]*gyro_ratio
            self._gy = struct.unpack('h', bytearray(ext_data[8:10]))
[0]*gyro_ratio
```

```
            self._gz = struct.unpack('h', bytearray(ext_data[10:12]))
[0]*gyro_ratio

            # 转化单位为uT
            mag_ratio = 800.0 / 32767.0
            self._mx = struct.unpack('h', bytearray(ext_data[12:14]))
[0]*mag_ratio
            self._my = struct.unpack('h', bytearray(ext_data[14:16]))
[0]*mag_ratio
            self._mz = struct.unpack('h', bytearray(ext_data[16:18]))
[0]*mag_ratio
        # 解析板子的姿态角
        # the attitude Angle of the board
        elif ext_type == self.FUNC_REPORT_IMU_EULER:
            self._roll = struct.unpack('f', bytearray(ext_data[0:4]))[0]
            self._pitch = struct.unpack('f', bytearray(ext_data[4:8]))[0]
            self._yaw = struct.unpack('f', bytearray(ext_data[8:12]))[0]
        # 解析IMU的四元数
        # the quaternion of IMU
        elif ext_type == self.FUNC_REPORT_IMU_QUAT:
            self._q0 = struct.unpack('f', bytearray(ext_data[0:4]))[0]
            self._q1 = struct.unpack('f', bytearray(ext_data[4:8]))[0]
            self._q2 = struct.unpack('f', bytearray(ext_data[8:12]))[0]
            self._q3 = struct.unpack('f', bytearray(ext_data[12:16]))[0]
        # 解析气压计数据
        elif ext_type == self.FUNC_REPORT_BARO:
            self._height = round(struct.unpack('f', bytearray(ext_data[0:4]))
[0], 2)
            self._temperature = round(struct.unpack('f',
bytearray(ext_data[4:8]))[0], 2)
            self._pressure = round(struct.unpack('f', bytearray(ext_data[8:12]))
[0], 5)
            self._pressure_contrast = round(struct.unpack('f',
bytearray(ext_data[12:16]))[0], 5)
        elif ext_type == self.FUNC_VERSION:
            self._version_H = struct.unpack('B', bytearray(ext_data[0:1]))[0]
            self._version_M = struct.unpack('B', bytearray(ext_data[1:2]))[0]
            self._version_L = struct.unpack('B', bytearray(ext_data[2:3]))[0]
        elif ext_type == self.FUNC_RETURN_STATE:
            self._rx_func = struct.unpack('B', bytearray(ext_data[0:1]))[0]
            self._rx_state = struct.unpack('B', bytearray(ext_data[1:2]))[0]
# 接收数据 receive data
    def _receive_data(self, data):
        if self.rx_flag == 0:
            if data == self._HEAD1:
                self.rx_flag = 1
        elif self.rx_flag == 1:
            if data == self._HEAD2:
                self.rx_flag = 2
            else:
                self.rx_flag = 0
        elif self.rx_flag == 2:
            self.data_len = data
            if self.data_len <= self._RX_MAX_LEN:
                self.rx_flag = 3
            else:
                self.data_len = 0
                self.rx_flag = 0
```

```
        elif self.rx_flag == 3:
            self.data_func = data
            self.rx_data = []
            self.rx_count = 4
            self.rx_flag = 4
        elif self.rx_flag == 4:
            self.rx_data.append(data)
            self.rx_count = self.rx_count + 1
            if self.rx_count >= self.data_len - 1:
                self.rx_flag = 5
        elif self.rx_flag == 5:
            self.rx_flag = 0
            rx_check_num = data
            check_sum = self._HEAD1 + self._HEAD2 + self.data_len +
self.data_func
            for a in self.rx_data:
                check_sum = a + check_sum
            check_sum = check_sum % 256
            if rx_check_num == check_sum:
                # print("parse_data", self.data_func, self.rx_data)
                self._parse_data(self.data_func, self.rx_data)
            else:
                if self._debug:
                    print("check sum error:", rx_check_num, check_sum)
                    print("data:", self.data_len, self.data_func, self.rx_data)
```

_receive_data: 接收数据，并调用parse_frame_data解析符合通信协议的数据。

_parse_frame_data()：解析数据。

3.读取imu数据

程序运行后，可以在下面的终端显示imu的数据。

USART.py

```python
import math
import struct
import time
from machine import UART, Pin


READ_INTERVAL = 0.1  # 传感器读取间隔（秒） Sensor read interval in seconds


class YbImuSerial(object):

    def __init__(self, port=0, debug=False, baudrate=115200, tx_pin=0, rx_pin=1):

        # 串口初始化（MicroPython UART） Serial port initialization for MicroPython UART
        self._dev = UART(port, baudrate=baudrate, tx=Pin(tx_pin), rx=Pin(rx_pin))  # 默认使用 UART0，TX=GP0，RX=GP

        self._debug = debug

        self._HEAD1 = 0x7E
        self._HEAD2 = 0x23
```

Shell

```
------ Sensor Data ------
Acceleration [g]:       x=-1.461, y= 0.680, z=-0.042
Gyroscope [rad/s]:      x=-4.322, y= 1.867, z=-2.431
Magnetometer [uT]:      x= 38.844, y=-17.408, z=-12.403
Quaternion:             w= 0.10231, x= 0.70284, y=-0.14816, z=-0.68573
Euler Angle [deg]:      roll= 95.25, pitch= 69.00, yaw=-87.43
Barometer:              height=-0.05 m, temperature= 41.15 °C
                        pressure= 101466.04688 Pa, pressure_diff= 101465.39844 Pa
-------------------------
```

MicroPython (Raspberry Pi Pico)

注意：以上为10轴IMU的数据读取，6轴无磁力计（Magnetometer）与气压计（Barometer）数据，9轴无气压计（Barometer）数据。